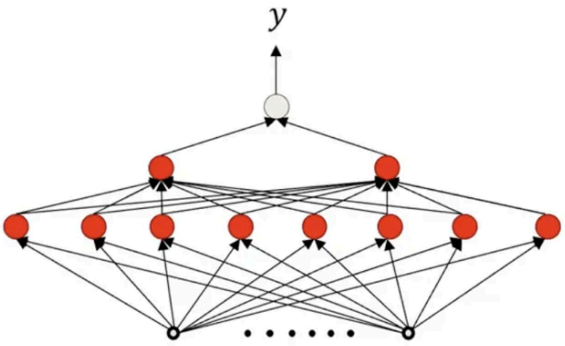
Lec8. Training Part VI

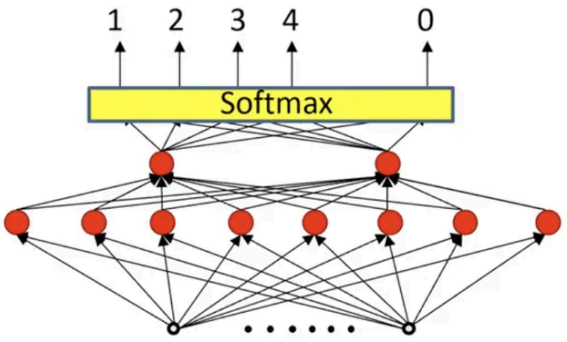
Generalization / Tricks of the trade

Training Neural Nets by Gradient Descent

Total training loss:

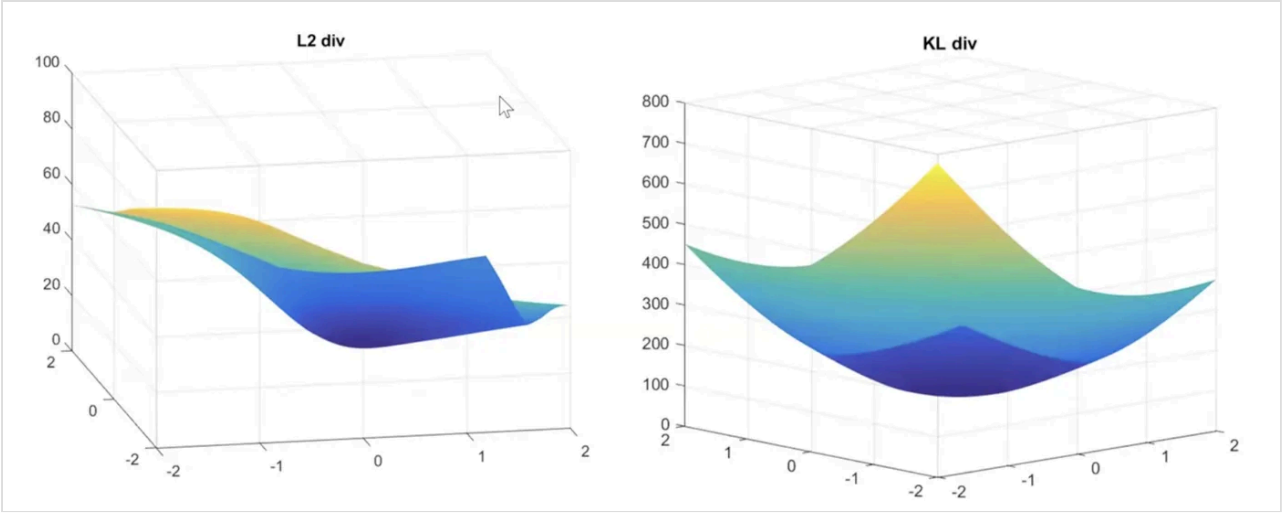
$$Loss = \frac{1}{T}\sum_t Div(\boldsymbol{Y_t}, \boldsymbol{d_t}; \mathbf{W}_1, \mathbf{W}_2, \ldots, \mathbf{W}_K)$$





Desired output:	d	Desired output: $[0,0,\ldots,1,\ldots,0]$
L2	$Div = \frac{1}{2}(y-d)^2$	$Div = \frac{1}{2}\sum_i (y_i - d_i)^2$
KL	$Div = -d\log(y) - (1-d)\log(1-y)$	$Div = \sum_i d_i \log(d_i) - \sum_i d_i \log(y_i)$

- 신경망 학습에서는 출력과 목표값의 특성에 따라 L2 loss(제곱오차), cross-entropy, KL divergence 등을 선택할 수 있다.
- 이진분류에서 $y \in (0,1)$, $d \in [0,1]$이면 binary cross-entropy는 $L_{CE} = -d\cdot\log y - (1-d)\cdot\log(1-y)$이다.
- Binary KL divergence는 $D_{KL}(d \,/\!/\, y) = d\cdot\log(d/y) + (1-d)\cdot\log((1-d)/(1-y))$이며, $0 \log 0 = 0$으로 정의한다.
- 두 함수는 d에만 의존하는 상수항만 다르므로 목표값을 고정하고 y를 최적화할 때 gradient가 같다.
- 특히 $d \in \{0,1\}$이면 두 식의 값도 일치한다.



- 다중분류에서 목표값 $d = (d_1, \cdots, d_C)$가 $d_k \geq 0$, $\Sigma_{k=1}^{C} d_k = 1$을 만족하는 확률벡터이고 출력 $y = \text{softmax}(z)$일 때, categorical cross-entropy $L_{CE} = -\Sigma_{k=1}^{C} d_k \log y_k$와 KL divergence $D_{KL}(d \,/\!/\, y) = \Sigma_{k=1}^{C} d_k \log(d_k/y_k)$는 logits z에 대해 볼록하며, 입력 특징이 고정된 선형 분류기의 마지막 층 가중치에 대해서도 볼록하다.
- 반면 sigmoid 출력에 제곱오차를 적용하면 logits에 대한 손실조차 비볼록할 수 있다.
- 그러나 다층 신경망에서는 logits가 가중치의 비선형 함수이므로, cross-entropy나 KL divergence를 사용하더라도 전체 가중치에 대한 손실함수가 일반적으로 볼록한 것은 아니다.
- 따라서 손실함수의 종류뿐 아니라 출력 activation과 모델 구조를 함께 고려해야 한다.
- 출력 activation은 손실함수와 합성되어 loss surface의 형태와 gradient에 영향을 준다.
- 예를 들어 $y = \sigma(z)$ 또는 $y = \text{softmax}(z)$이고 $z = f_W(x)$라면, 파라미터 W에 대한 loss surface는 $L(W) = \ell(y(f_W(x)), d)$로 결정된다.
- 따라서 activation만으로 loss surface를 판단할 수 없으며, 출력 activation과 손실함수의 조합, 그리고 네트워크 구조를 함께 분석해야 한다.

A note on derivatives

$$\nabla_z \frac{1}{2} \|y - d\|^2 = (y - d)^\top$$

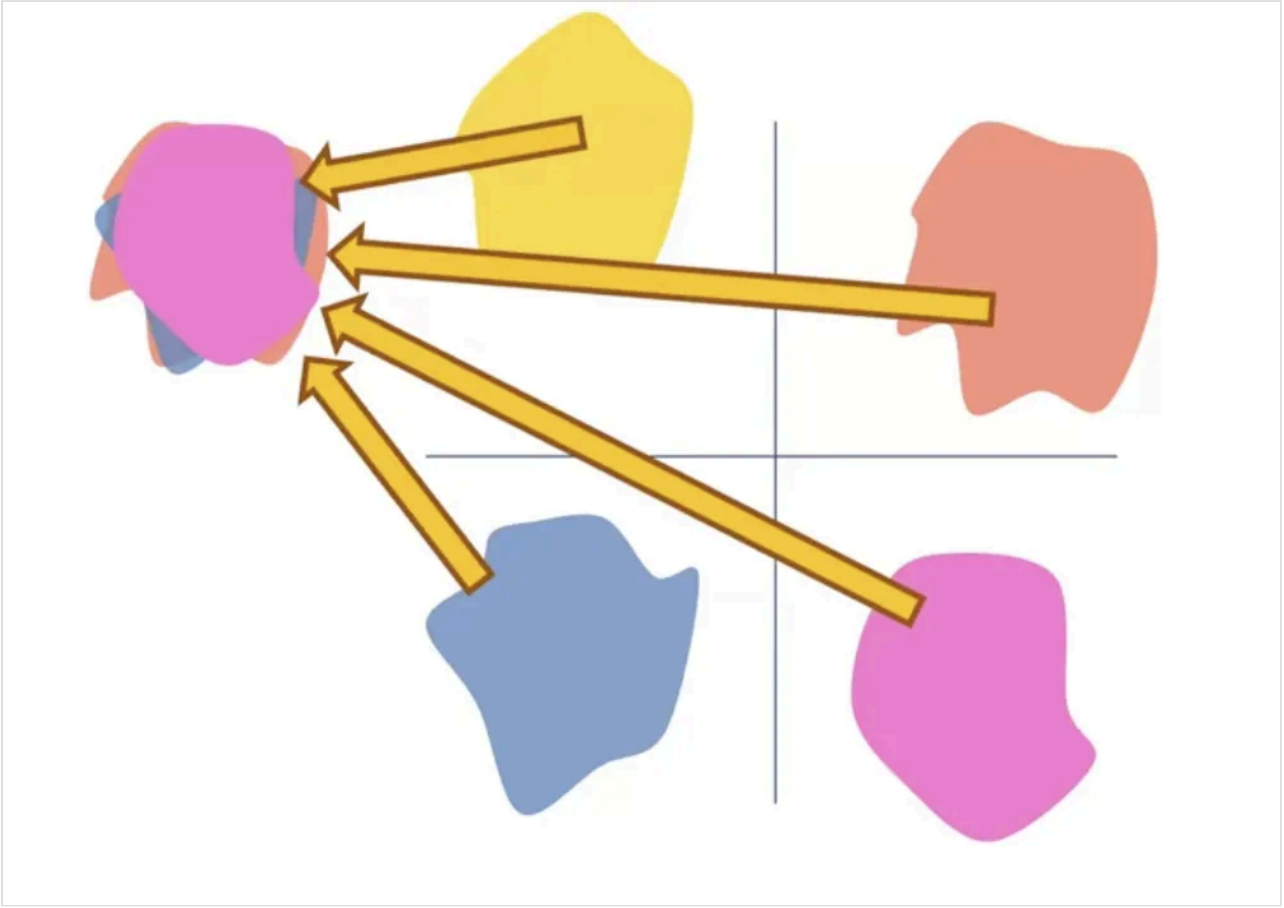
$$\nabla_z KL(y, d) = (y - d)^\top$$

- Backpropagation에서 필요한 것은 손실함수를 출력 이전의 logit z에 대해 미분한 값이다.
- 선형 출력 $y = z$와 L2 loss $L = (1/2) \,/\!/\, y-d \,/\!/\, ^2$를 결합하면 $\nabla_z L = y-d$이다.

- 또한 $y = \mathrm{softmax}(z)$이고 목표값 d가 $d_k \geq 0$, $\Sigma_{k=1}^{C} d_k = 1$을 만족하는 확률벡터이면, categorical cross-entropy 또는 $D_{KL}(d \,/\!/\, y)$를 사용할 때 $\nabla_z L = y - d$가 된다. 이는 softmax–loss 조합에서 목표 확률벡터의 성분 합이 1이라는 조건을 이용한 결과이며, 이진분류의 sigmoid–binary cross-entropy 조합에서도 logit에 대한 도함수가 $y-d$로 단순화된다.
- 그러나 sigmoid 출력과 L2 loss를 결합하면 $\partial L / \partial z = (y-d)y(1-y)$이므로 일반적으로 같지 않다.
- 따라서 도함수가 $y-d$로 단순화되는 것은 특정 activation–loss 조합의 성질이며, L2와 KL의 학습 과정이 일반적으로 동일하다는 의미는 아니다.

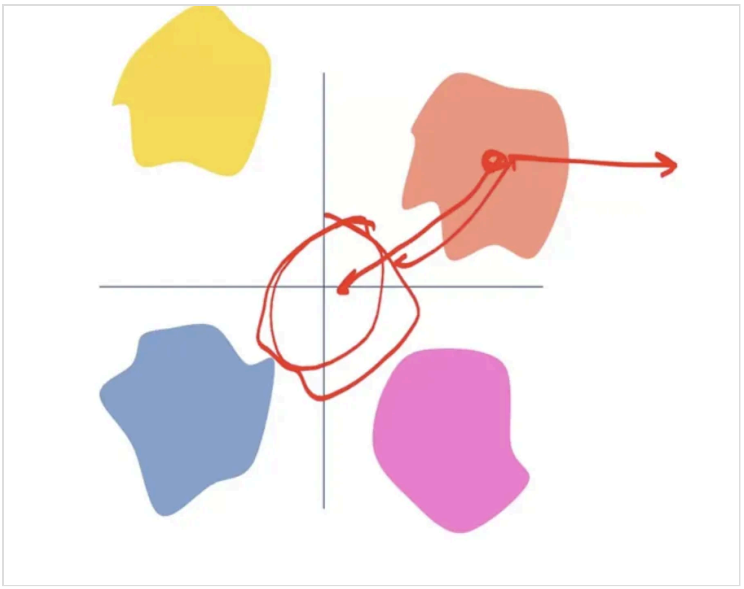
The problem of covariate shifts

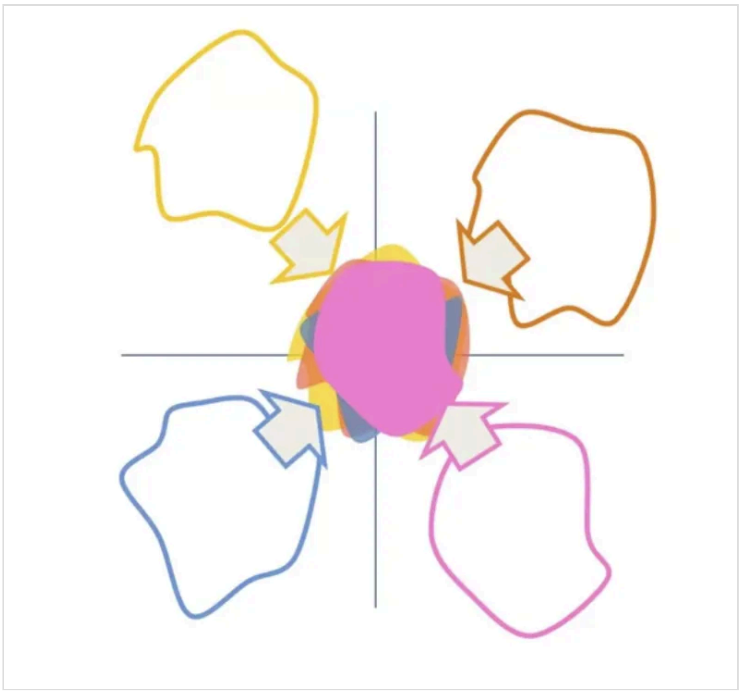
- 일반적인 covariate shift는 학습 분포와 평가 분포 사이에서 입력의 주변 분포가 $P_{train}(X) \neq P_{test}(X)$로 달라지지만, 공통 입력 영역에서 조건부 레이블 분포 $P_{train}(Y|X) = P_{test}(Y|X)$는 유지된다고 가정하는 분포 변화다. 반면 internal covariate shift는 학습 중 앞쪽 층의 파라미터가 갱신되면서 뒤쪽 층이 받는 activation의 분포가 변하는 현상을 가리키는 용어이며, 일반적인 covariate shift와는 구별해야 한다.
- Mini-batch마다 표본평균과 표본분산이 다른 것은 표본 추출에 따른 변동일 수도 있으므로 두 개념을 동일시해서는 안 된다.
- Batch Normalization은 각 mini-batch의 activation을 정규화하여 학습을 안정화하는 방법이며, 그 효과는 단순한 분포 변화 억제뿐 아니라 gradient와 최적화 지형의 성질과도 관련된다.

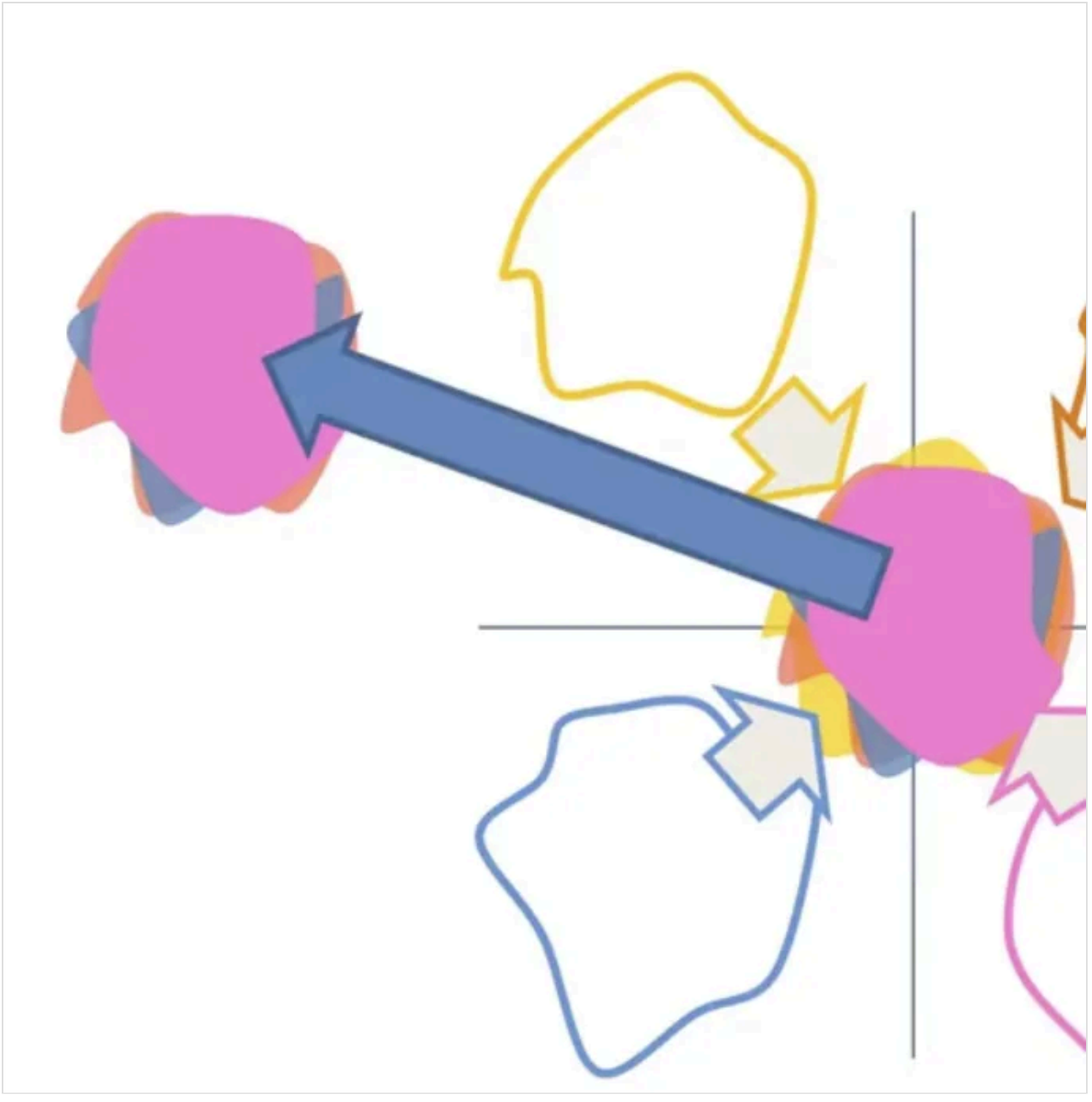


- Batch Normalization은 mini-batch의 각 특징에 대해 평균을 빼서 중심을 0으로 옮기고, $\sqrt{(\sigma_B^2 + \epsilon)}$으로 나누어 분산의 크기를 정규화한다.

Move all minibatch to a “standard” location



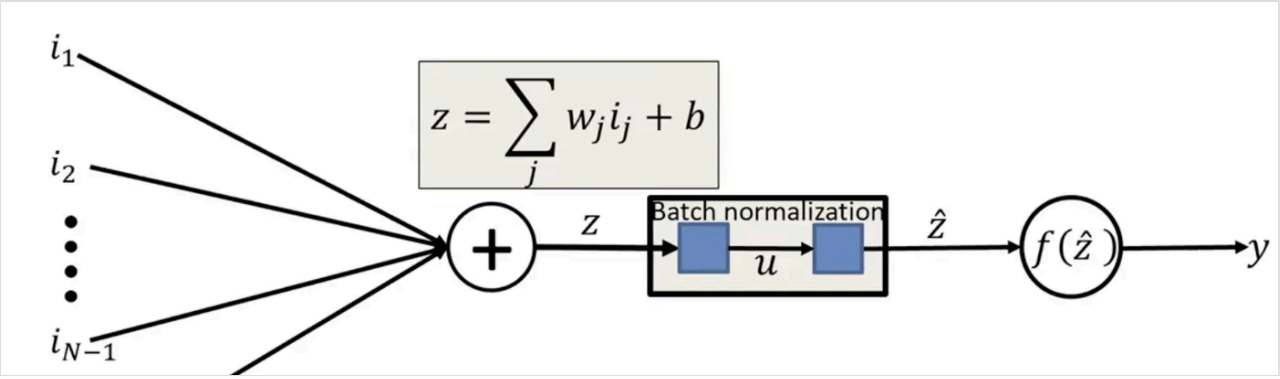




- 1. 원점으로 이동

- 2. 표준편차로 나눠서 분산을 같게 함
- 3. 고정된 위치로 이동

Batch normalization : Training

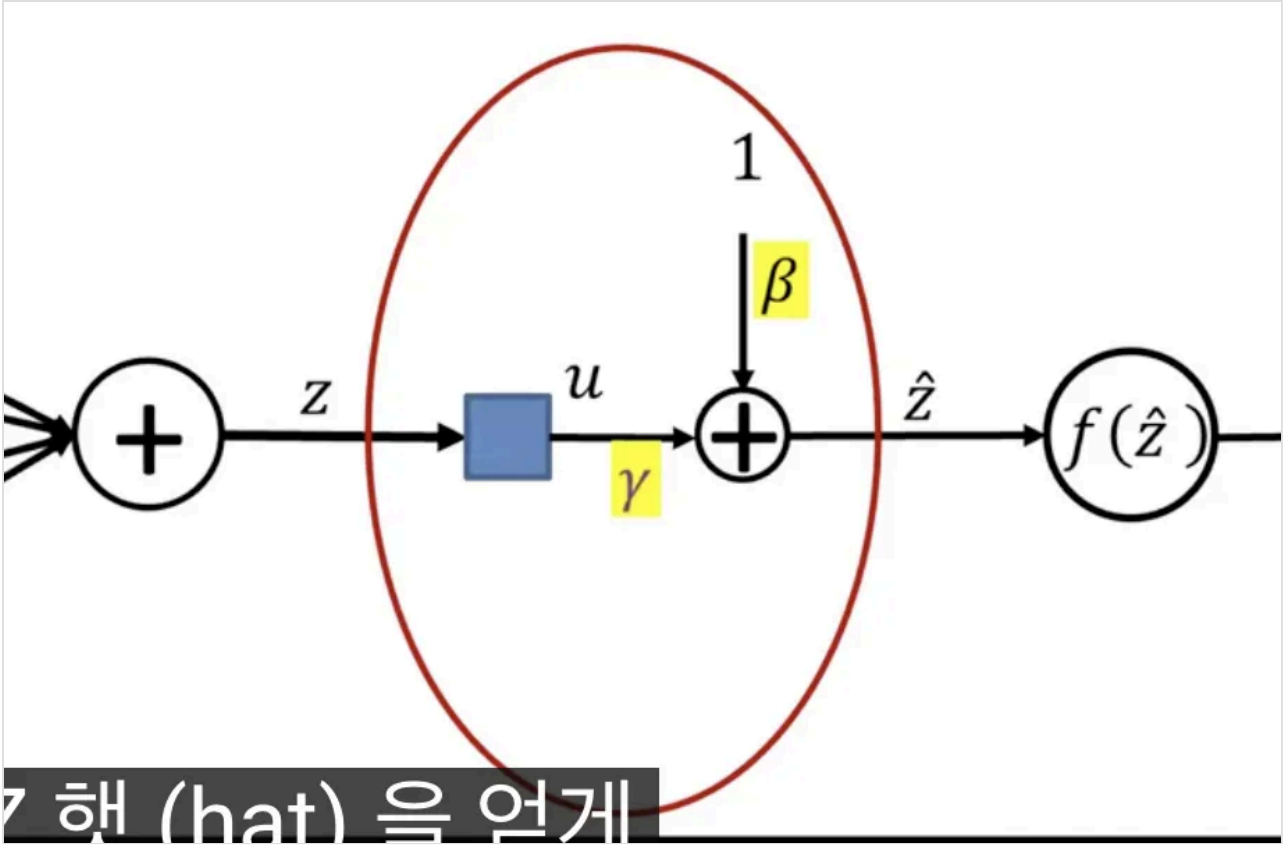


$$\mu_B = \frac{1}{B}\sum_{i=1}^{B} z_i \qquad \sigma_B^2 = \frac{1}{B}\sum_{i=1}^{B}(z_i - \mu_B)^2$$

$$u_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \qquad \hat{z}_i = \gamma u_i + \beta$$

- B: minibatch size
- μ_B : Minibatch mean
  σ : Minibatch standard deviation

- 이어서 학습 가능한 γ와 β를 사용해 출력을 affine 변환한다.
- 이 과정은 mini-batch 간 평균과 분산의 차이를 줄일 수 있지만 분포의 모든 형태를 동일하게 만드는 것은 아니며, 정규화 직후의 분산도 일반적으로 $\sigma_B^2/(\sigma_B^2+\epsilon)$이다.
- ⇒ **“Move all minibatch to a “standard” location” 부분이랑 같은 것을 하는 거임.(위에)**



Batch Normalization : Scale & Shift

- $\hat{z}_i = \gamma u_i + \beta$
  - **γ (Gamma)** : Scale → 분포의 크기 조절
    β (Beta) : Shift → 분포의 위치 이동
  - Batch Normalization은 $u_i = (z_i - \mu_B)/\sqrt{(\sigma_B^2+\epsilon)}$으로 정규화한 뒤 $\hat{z}_i = \gamma u_i + \beta$를 적용한다.
  - 정규화된 u의 평균은 0이고 분산은 $\sigma_B^2/(\sigma_B^2+\epsilon)$이다.
  - Scale 파라미터 γ와 shift 파라미터 β를 적용하면 $\hat{z}$의 평균은 β, 분산은 $\gamma^2\sigma_B^2/(\sigma_B^2+\epsilon)$가 된다.
  - 따라서 γ, β는 정규화된 활성값의 크기와 위치를 학습 가능한 방식으로 조절하며, 모델이 표현에 필요한 scale과 shift를 선택하도록 한다.
- ⇒ **Batch Normalization = 정규화 + Scale & Shift**

Batch Normalization : Gradient 계산

$$Loss(minibatch) = \frac{1}{B}\sum_t Div(Y_t(X_t, \mu_B, \sigma_B^2), d_t(X_t))$$

1. Without BN

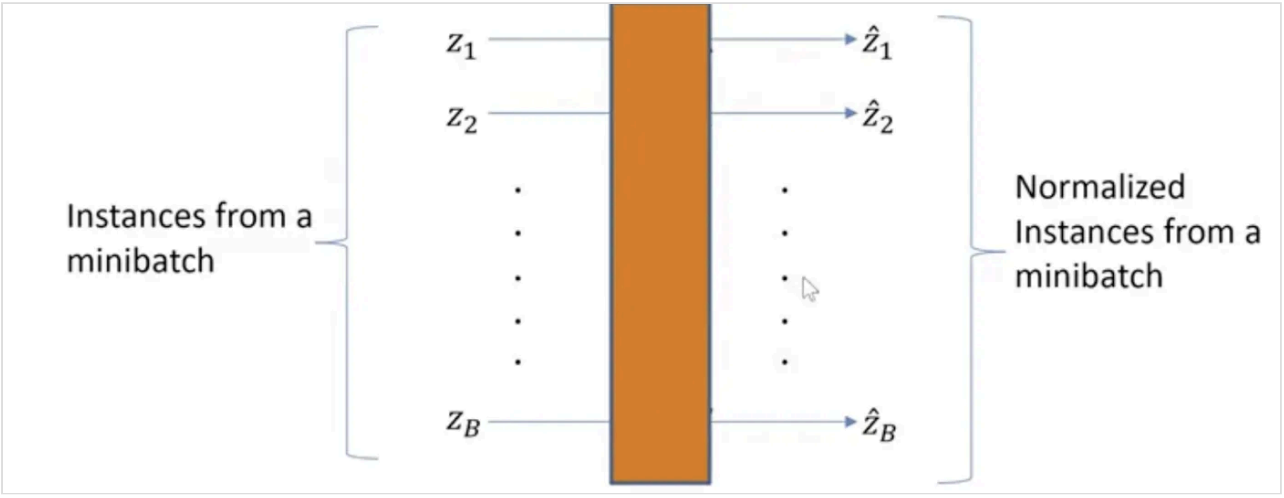
- Loss = (1/B) Σ_i L_i
  ∇Loss = (1/B) Σ_i ∇L_i

- BN처럼 batch 내 샘플을 결합하는 연산이 없고, 각 샘플의 순전파가 독립적이며, 전체 손실이 $L = (1/B)\,\Sigma_i L_i$로 분리되는 경우 각 L_i와 개별 gradient $\nabla_\theta L_i$는 고정된 파라미터 θ 아래에서 다른 샘플의 입력값에 의존하지 않는다.
- 이 경우 $\nabla_\theta L = (1/B)\,\Sigma_i \nabla_\theta L_i$가 된다.
- 그러나 BN이나 샘플 간 비교를 사용하는 손실함수가 있다면 개별 출력 또는 손실이 다른 샘플에 의존할 수 있다.

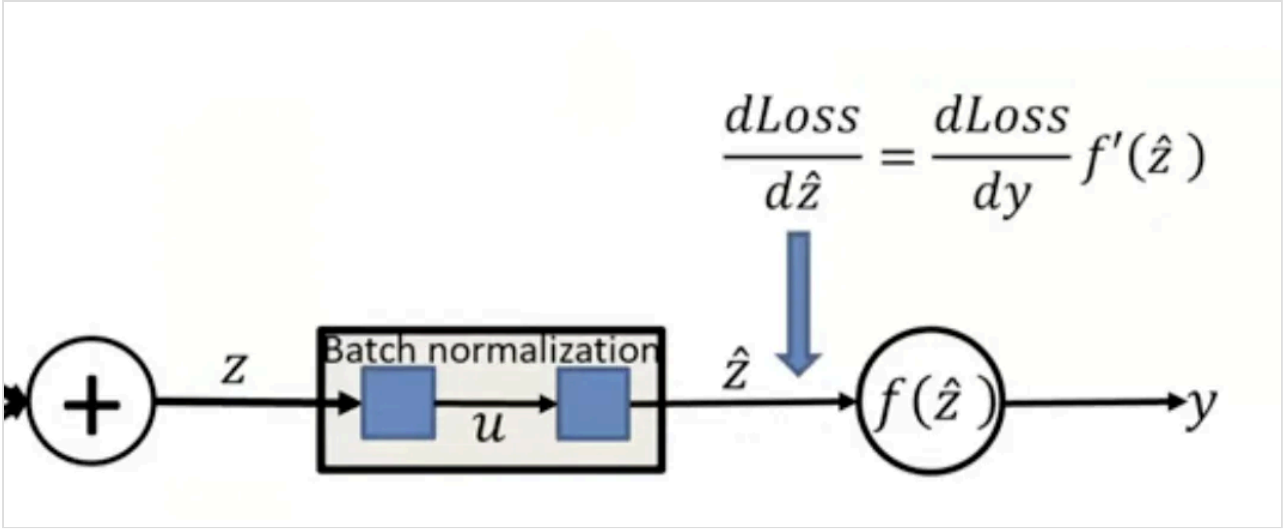
2. With BN(BatchNorm)

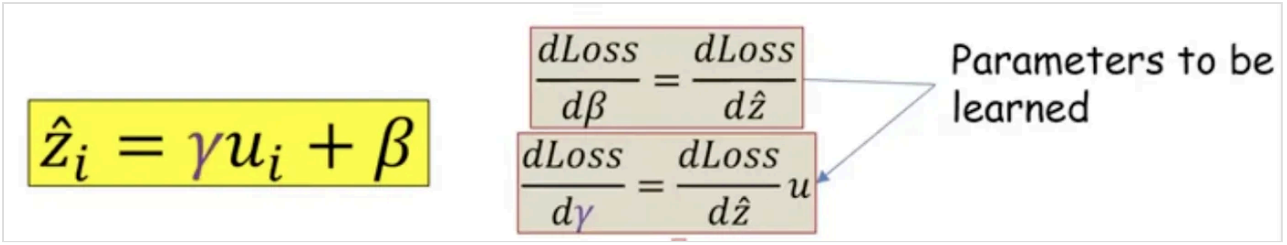
- 완전연결층의 Batch Normalization에서는 각 특징 차원마다 mini-batch에 속한 B개 값 $z_1, \cdots, z_B$에 대해 $\mu_B = (1/B)\,\Sigma_{i=1}^{B} z_i$, $\sigma_B^2 = (1/B)\,\Sigma_{i=1}^{B} (z_i - \mu_B)^2$를 계산하고, $u_i = (z_i - \mu_B)/\sqrt{(\sigma_B^2+\epsilon)}$으로 정규화한다.
- 여기서 $B \geq 1$, $\epsilon > 0$이다.
- CNN에서 일반적으로 사용하는 공간적 BN은 채널별로 batch 및 공간 위치에 걸쳐 평균과 분산을 계산한다.
- $Loss = \frac{1}{B}\Sigma_i \; Div(Y_i \; (X_i \; , \mu_B, \sigma^2 \; _B), d_i \;)$
- → $\mu_B, \sigma^2 \; _B$가 모든 데이터에 의해 결정됨.
- → 각 데이터의 출력과 Loss가 **Batch 내 다른 데이터에도 의존**하게 됨.
- → Backpropagation 시 다른 데이터의 영향까지 고려해야 함.
- **⇒ BN은 Mini-batch 내 데이터 간 의존성을 만들어 Gradient 계산을 복잡하게 함.**
- BN의 affine 변환은 $\hat{z}_i = \gamma u_i + \beta$이며, γ, β는 같은 특징 차원의 모든 mini-batch 샘플이 공유한다.
- 전체 batch 손실을 L, 상류 gradient를 $\delta_i = \partial L/\partial \hat{z}_i$라고 두면 $\partial L/\partial \gamma = \Sigma_{i=1}^{B} \delta_i u_i$, $\partial L/\partial \beta = \Sigma_{i=1}^{B} \delta_i$이다.
- 각 샘플에서 얻은 국소 미분 기여분은 공유 파라미터에 대해 모두 합산해야 하며, δ_i에 손실 평균의 1/B가 이미 포함되어 있다면 추가로 나누지 않는다.

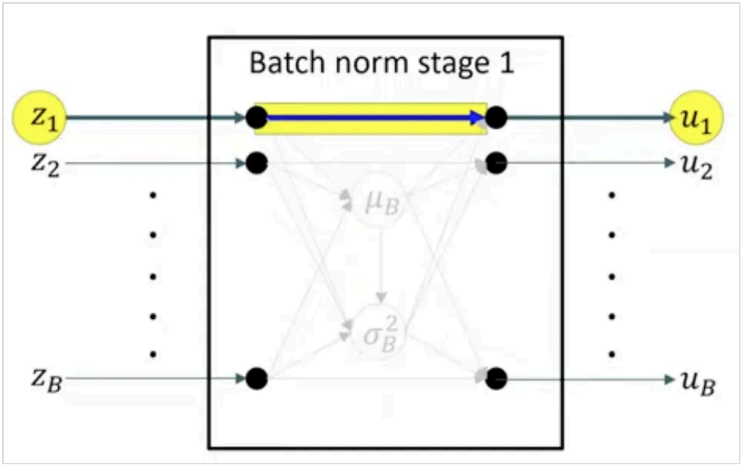
Batchnorm is a vector function over the minibatch

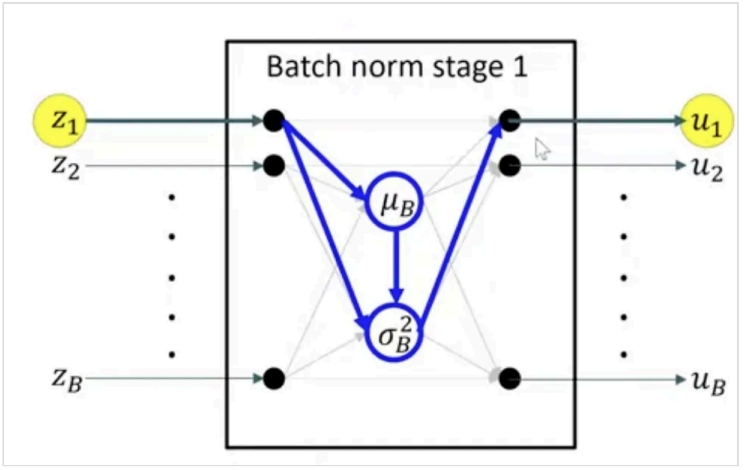


- Batch Normalization 는 vector function과 다를 게 없음.
  - → 왜냐하면, 모든 z가 $\hat{z}$에 영향을 주게 됨

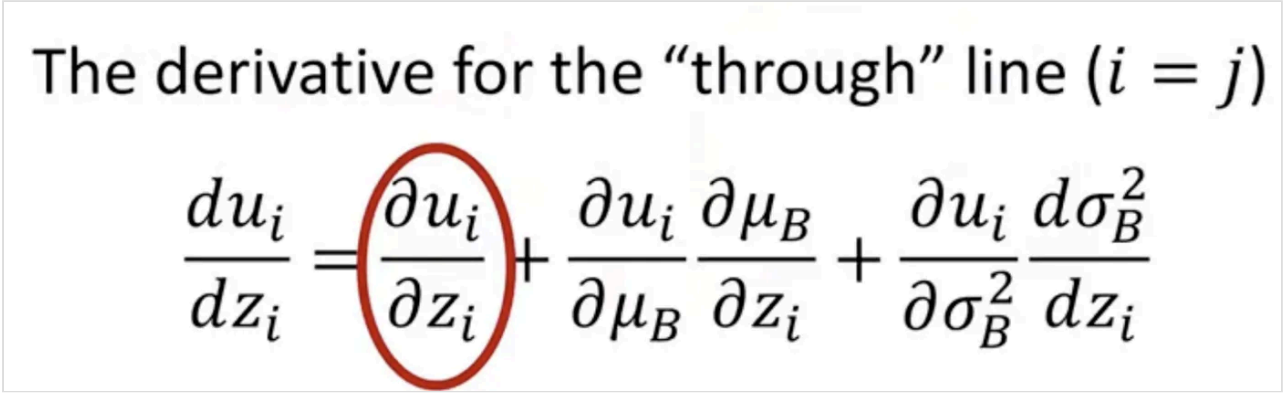




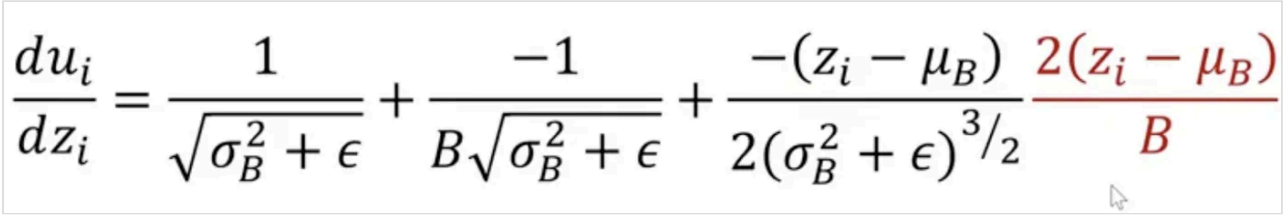




- 각 z에 대해서, μ, σ 모두 바뀌며, 이에 의해 u가 바뀜.



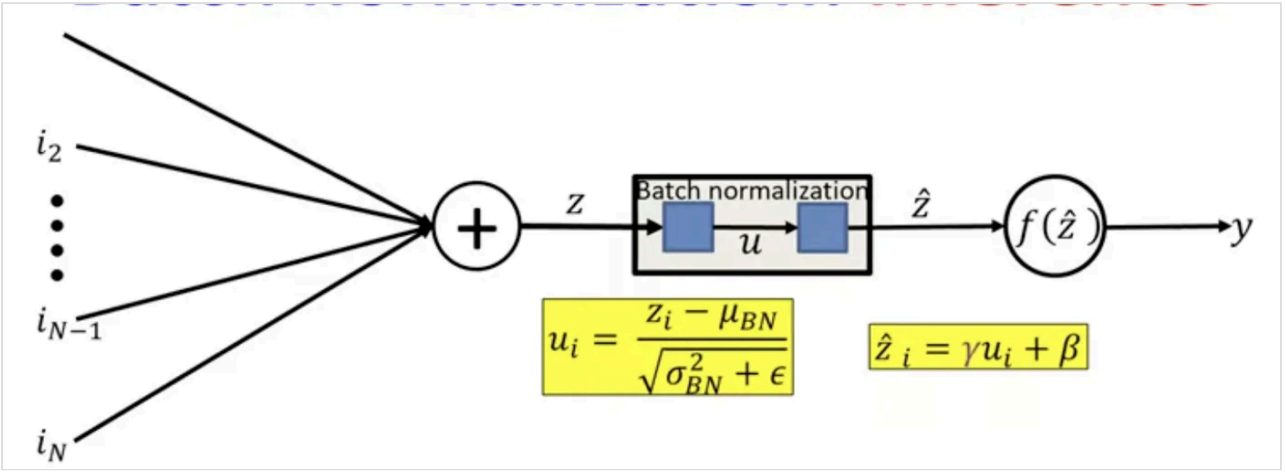
- 따라서, 이와 같이 $\frac{du_i}{dz_i}$가 되게 됨.



- 각 미분결과를 넣으면 → 위와 같은 방정식을 구할 수 있음.
- BN의 Jacobian 유도: $u_i = (z_i - \mu_B)/s$, $s = \sqrt{(\sigma_B^2 + \epsilon)}$라고 두자.
- 모든 z_j는 μ_B와 σ_B^2에 영향을 주므로 u_i는 자신의 입력뿐 아니라 batch의 다른 입력에도 의존한다.
- 먼저 $\partial \mu_B / \partial z_j = 1/B$, $\partial \sigma_B^2 / \partial z_j = 2(z_j - \mu_B)/B$를 구한다.
- 다음으로 chain rule을 적용하면 $\partial u_i / \partial z_j = (\delta_{ij} - 1/B)/s - (z_i - \mu_B)(z_j - \mu_B)/(B \cdot s^3)$을 얻는다.
- 여기서 $\delta_{ij} = 1$은 i = j일 때이고, 그렇지 않으면 0이다.
- 핵심: i = j에서는 직접 미분 경로가 추가되지만, i ≠ j에서도 평균과 분산을 통한 간접 미분 경로가 존재한다.
- 따라서 BN의 Jacobian은 일반적으로 대각행렬이 아니며, Backpropagation에서는 다른 샘플에서 전달되는 gradient도 합산해야 한다.

$$\frac{du_j}{dz_i} = \begin{cases} \frac{1}{\sqrt{\sigma_B^2 + \epsilon}} + \frac{-1}{B\sqrt{\sigma_B^2 + \epsilon}} + \frac{-(z_i - \mu_B)^2}{B(\sigma_B^2 + \epsilon)^{3/2}} & \text{if } j = i \\ \frac{-1}{B\sqrt{\sigma_B^2 + \epsilon}} + \frac{-(z_i - \mu_B)(z_j - \mu_B)}{B(\sigma_B^2 + \epsilon)^{3/2}} & \text{if } j \neq i \end{cases}$$

Batch normalization : Inference



$$\mu_{BN} = \frac{1}{Nbatches} \sum_{batch} \mu_B(batch)$$

$$\sigma^2_{BN} = \frac{B}{(B-1)Nbatches} \sum_{batch} \sigma^2_B(batch)$$

Training

- 현재 Minibatch의 평균 μ_B, 분산 σ^2_B 사용.
  Minibatch가 바뀌면 정규화 기준도 달라짐.

Inference

- Batch Normalization은 Training에서 현재 mini-batch의 평균 μ_B와 분산 σ_B^2를 사용하지만, 일반적인 Inference에서는 학습 중 추정해 저장한 running mean μ_{run}과 running variance σ_{run}^2를 고정하여 사용한다.
- 따라서 추론식은 $\hat{z}_i = \gamma(z_i - \mu_{run})/\sqrt{(\sigma_{run}^2+\epsilon)} + \beta$이다.
- Running 통계량은 구현에 따라 이동평균이나 별도 집계 방식으로 추정할 수 있다.
- 동일한 크기 B > 1의 mini-batch를 동일한 모집단에서 독립적으로 표본 추출했다면 B/(B−1)을 이용한 batch 분산의 불편추정이 가능하지만, 이렇게 평균한 값이 임의로 구성된 전체 데이터의 정확한 분산과 항상 일치하는 것은 아니다.
- Inference에서는 정규화 통계량과 학습된 γ, β를 고정하므로 개별 샘플을 다른 추론 샘플과 독립적으로 처리할 수 있다.

$$\hat{z}_i = \gamma \frac{z_i - \mu_{BN}}{\sqrt{\sigma^2_{BN} + \epsilon}} + \beta$$

- ⇒ **Training에서는 Minibatch별 통계량을 사용하지만, Inference에서는 학습 중 저장한 통계량을 고정해서 사용함.**
- ⇒ 따라서 Inference에서는 **다른 데이터에 의존하지 않고 개별 데이터를 예측할 수 있음.**

Batch Normalization : Application & Benefits

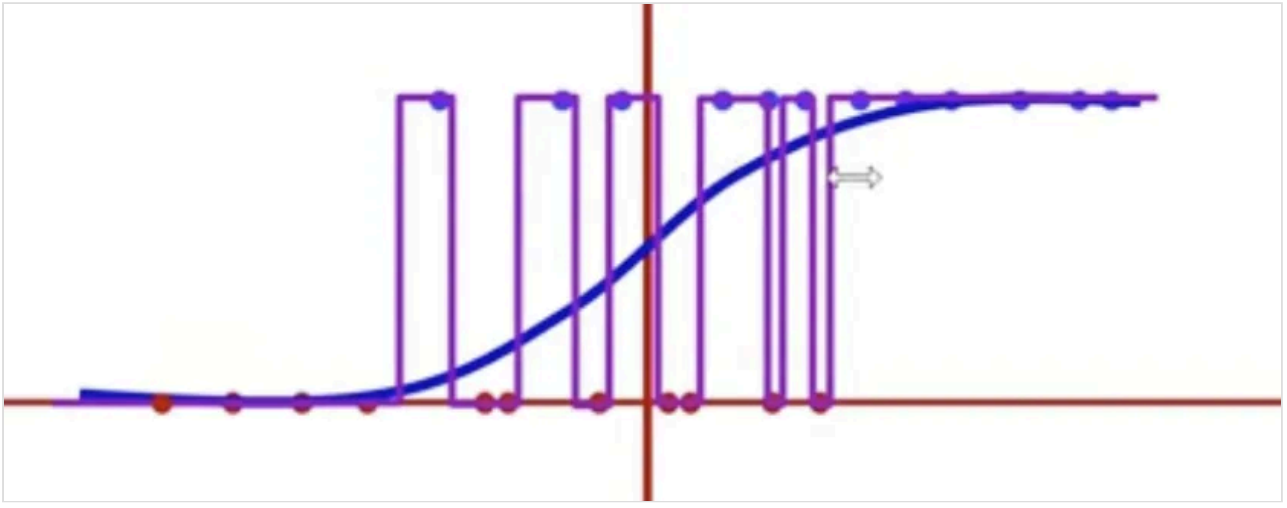
1. 적용 위치

- 일반적으로 Wx + b → BN → Activation 순서
  모든 Layer에 적용할 필요는 없음

2. 효과

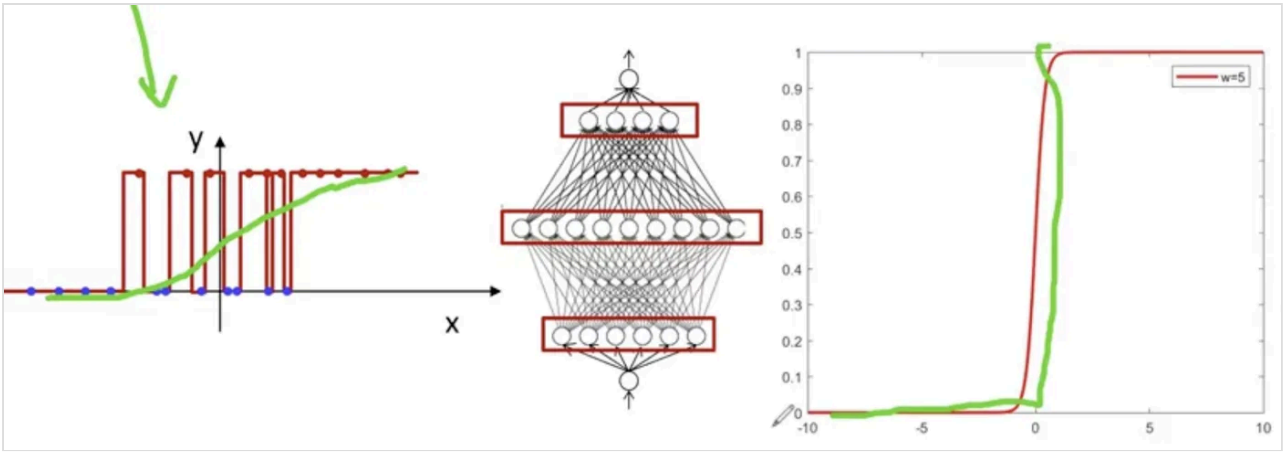
- **Faster Convergence : 학습 안정화 → 더 큰 Learning Rate 사용 가능**
  Better Generalization : Regularization 효과로 성능 개선 가능
  Less Dropout : BN이 일부 Regularization 역할 수행
  그러나 learning rate decay의 적절한 속도는 BN 사용 여부만으로 결정되지 않으며, optimizer와 검증 성능에 따라 조정해야 한다.
  일반적인 무작위 표본 기반 학습에서는 데이터를 shuffle하여 mini-batch 간 구성 편향을 줄이는 것이 도움이 될 수 있지만, 순서 정보가 중요한 학습에서는 데이터의 시간적·구조적 제약을 고려해야 한다.
- Batch Normalization은 gradient의 안정성을 높여 더 큰 learning rate를 사용할 수 있게 하거나 학습 수렴을 빠르게 만드는 경우가 있다.
- BN의 수렴 및 일반화 효과 역시 모델과 학습 조건에 따라 달라진다.
- Optimization 효과: Batch Normalization은 학습을 더 안정적으로 만들고 경우에 따라 더 큰 learning rate를 허용하여 수렴 속도를 향상시킨다. 다만 모든 모델에서 수렴이 빨라지는 것은 아니다.
- Regularization 및 Generalization 효과: Training 과정에서 mini-batch 통계량이 달라지는 것은 일부 정규화 효과를 제공할 수 있으며, 이에 따라 일반화 성능이 향상되거나 다른 정규화 기법의 필요성이 감소하는 경우가 있다.
- 학습 설정 시 고려사항: Learning rate schedule은 검증 성능에 맞추어 조정해야 한다. Mini-batch 구성은 BN 통계량에 영향을 주므로, 일반적인 무작위 표본 기반 학습에서는 적절한 shuffling이 도움이 될 수 있다. Batch size가 너무 작으면 통계량 추정의 변동성이 커질 수 있으므로 batch 크기도 고려해야 한다.
- ⇒ **BN은 학습을 안정화하고 수렴 속도와 일반화 성능을 개선하는 데 도움을 줌.**

Overfitting and Smoothing



- 정답: 파란색 함수
  이 상황에서, 보라색으로 학습되는 것을 막기 힘듦.

- 방지하려면? → 왜 이게 학습되는지를 봐야 함.



- 일부 신경망에서는 가중치의 크기가 커질수록 출력 함수가 더 급격하게 변하거나 학습 데이터에 지나치게 민감한 함수를 표현할 수 있다.
- 예를 들어 sigmoid σ(wx)의 전이 구간은 |w|가 커질수록 가파르게 변한다.
- L2 정규화는 손실함수에 $(\lambda/2)\ \Sigma_k\ \#W_k\#_F^2$를 추가하여 큰 가중치에 불이익을 주는 방법이다.
- 이를 통해 함수의 복잡성이나 극단적인 가중치 증가를 억제하고 과적합을 완화할 수 있지만, 모든 종류의 과적합을 반드시 방지하는 것은 아니다.

$$L(W_1, W_2, \ldots, W_K) = \frac{1}{T}\sum_t Div(Y_t, d_t; W_1, W_2, \ldots, W_K) + \frac{1}{2}\lambda \sum_k \|W_k\|_F^2$$

- = 정규화항을 두면 됨.

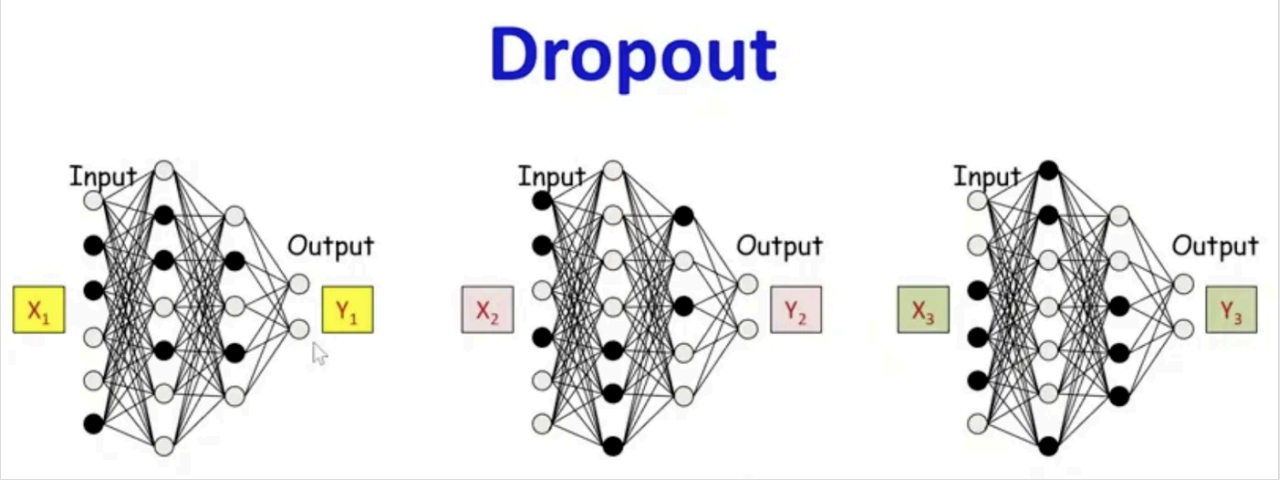
$$W_k \leftarrow W_k - \eta \nabla_{W_k} L^T = W_k - \eta \frac{1}{b} \sum_{t}^{t+b-1} \nabla_{W_k} Div(Y_t, d_t)^T - \eta\lambda W_k$$

$$W_k \leftarrow (1-\beta)W_k - \eta \frac{1}{b} \sum_{\tau=t}^{t+b-1} \nabla_{W_k} Div(Y_t, d_t)$$

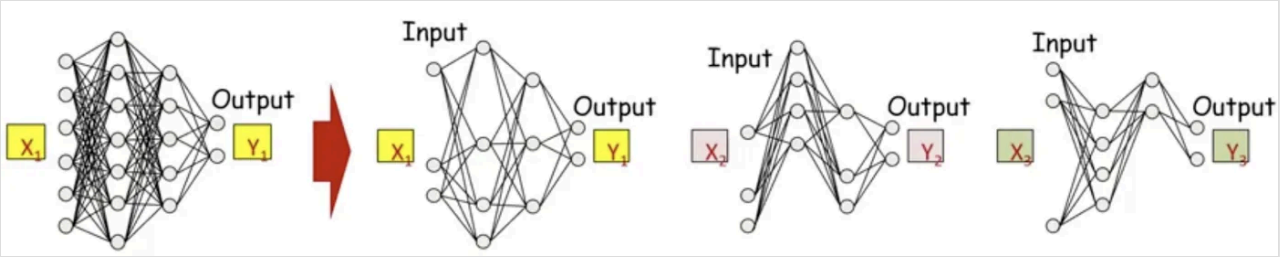
- L2 정규화가 없는 SGD의 파라미터 갱신은 $W_k \leftarrow W_k - \eta g_k$이다.
- 손실함수에 $(\lambda/2)\ \#W_k\#_F^2$를 추가하면 $\nabla_{W_k} L = g_k + \lambda W_k$가 되므로 $W_k \leftarrow W_k - \eta(g_k + \lambda W_k) = (1-\eta\lambda)W_k - \eta g_k$로 갱신된다.
- $\beta = \eta\lambda$로 두면 $W_k \leftarrow (1-\beta)W_k - \eta g_k$이며, 정규화가 없을 때 W_k 앞의 계수는 1이다.
- 이처럼 L2 정규화가 추가된 SGD에서는 매 단계 가중치를 줄이는 weight decay 효과가 나타난다.

Depth와 표현력

- 신경망의 깊이가 증가하면 여러 비선형 변환을 합성할 수 있으므로 특정한 복잡한 함수를 상대적으로 적은 뉴런 또는 파라미터로 효율적으로 표현할 수 있다.
- 따라서 같은 정도의 모델 자원을 사용하더라도 더 깊은 구조가 유리한 함수와 데이터가 존재한다.
- 그러나 깊이가 증가한다고 학습 정확도나 일반화 성능이 항상 향상되는 것은 아니며, 최적화의 어려움과 데이터 특성을 함께 고려해야 한다.



> **Dropout** : 학습하면서, 뉴런을 일부러 비활성화 함.



- Dropout은 Training 과정에서 미리 설정한 유지 확률 p에 따라 뉴런의 출력을 무작위로 0으로 만드는 정규화 기법이다.
- 어떤 뉴런이 중요한지 판단해 선택적으로 제거하는 것은 아니며, 비활성화된 뉴런에 연결된 경로가 해당 학습 단계에서 기여하지 않도록 한다.
- 매 학습 단계마다 서로 다른 부분 네트워크가 활성화되고 이들이 파라미터를 공유하면서 학습하므로 특정 뉴런 간의 과도한 공동 적응을 줄일 수 있다.
- 일반적인 Dropout은 네트워크 구조 자체를 영구적으로 학습하거나 변경하는 기법이 아니다.



Input: D dimensional vector $\mathbf{x} = [x_j,\ j = 1 \ldots D]$

Set:

- $D_0 = D$, is the width of the 0th (input) layer
- $y_j^{(0)} = x_j,\ j = 1 \ldots D; \quad y_0^{(k=1\ldots N)} = x_0 = 1$

For layer $k = 1 \ldots N$

Mask takes value 1 with prob. α, 0 with prob $1 - \alpha$

- $mask(k-1, j) = Bernoulli(\alpha)\ ,\ j = 1 \ldots D_{k-1}$
- $y_j^{(k-1)} = y_j^{(k-1)}.mask(k-1, j),\ j = 1 \ldots D_{k-1}$
- For $j = 1 \ldots D_k$
  - $z_j^{(k)} = \sum_{i=0}^{N_k} w_{i,j}^{(k)} y_i^{(k-1)} + b_j^{(k)}$
  - $y_j^{(k)} = f_k\left(z_j^{(k)}\right)$

Output:

- $Y = y_j^{(N)}, j = 1..D_N$

> 일반적인 신경망에서는 일부 뉴런이 다른 특정 뉴런의 출력에 과도하게 의존하는 공동 적응(co-adaptation)이 발생할 수 있다.

- Dropout은 학습 중 뉴런의 출력을 무작위로 제거하여 특정 뉴런 조합에만 의존하는 것을 어렵게 만들고, 다양한 부분 네트워크에서도 유용한 표현을 학습하도록 유도한다.
- 이를 통해 과적합을 완화할 수 있지만, 모든 feature 중복을 제거하거나 공동 적응을 완전히 방지하는 것은 아니다.

Training

- 뉴런을 무작위로 비활성화
  → N개 뉴런에 대해 최대 2^N개의 Sub-network를 학습하는 효과

Inference

- Dropout은 학습 중 무작위 mask에 의해 생성되는 여러 부분 네트워크가 파라미터를 공유하도록 학습하며, 추론 시에는 이들의 예측 결합 효과를 하나의 네트워크로 효율적으로 근사한다.
- 비선형 함수에서는 일반적으로 E[f(Y)] ≠ f(E[Y])이므로, 뉴런 출력의 기대값을 사용하는 스케일링 방식이 모든 부분 네트워크의 예측 평균을 정확히 계산하는 것은 아니다.
- 특히 단일 네트워크의 출력이 부분 네트워크 예측의 산술평균과 항상 일치하지 않으며, 원래 Dropout의 모델 결합 해석에는 확률 예측의 정규화된 기하평균에 대한 근사도 포함된다.

- 따라서 Dropout의 inference scaling은 일반적으로 정확한 앙상블 평균이 아니라 효율적인 근사 방법으로 이해해야 한다.

실제 구현

- 유지 확률을 p, Dropout mask를 D ~ Bernoulli(p), Dropout 적용 전 활성값을 a라고 하자.
- Standard Dropout은 Training에서 a_{train} = Da를 사용하고 Inference에서는 a_{test} = pa를 사용한다.
- 이는 다음 층의 해당 활성값에 연결되는 가중치를 p배 하는 것과 동등하게 구현할 수 있지만, 모든 가중치와 bias를 일괄적으로 스케일링한다는 의미는 아니다.
- Inverted Dropout은 $0 < p \leq 1$에서 Training 중 a_{train} = Da/p로 보정하고 Inference에서는 a_{test} = a를 사용한다.
- 두 방식 모두 학습·추론 단계에서 해당 활성값의 기대 크기를 일치시키는 것이 목적이며, 전체 비선형 네트워크의 예측 기댓값까지 정확히 보존한다는 의미는 아니다.
- ⇒ **Dropout은 여러 Sub-network의 Ensemble 효과를 하나의 네트워크로 근사하는 방법임.**
  - 1. Dropout의 정의: Dropout은 학습 중 유지 확률 p에 따라 각 뉴런의 출력을 무작위로 0으로 만들어 공동 적응을 완화하는 정규화 기법이다.
  - 2. Training: D ~ Bernoulli(p)를 사용하여 각 뉴런의 출력에 mask를 적용한다. Standard Dropout은 a_{train} = Da이며, Inverted Dropout은 a_{train} = Da/p이다.
  - 3. Inference: Standard Dropout에서는 a_{test} = pa로 출력 크기를 보정한다. Inverted Dropout에서는 학습 시 미리 보정했으므로 a_{test} = a를 사용한다.
  - 4. 기댓값: Standard Dropout은 E[Da] = pa, Inverted Dropout은 E[Da/p] = a를 만족한다. 이 등식은 Dropout이 적용되는 개별 활성값에 대한 기대값 관계다.
  - 5. Ensemble 해석: 매번 다른 mask가 만들어내는 부분 네트워크는 파라미터를 공유하며 학습된다. 추론 시에는 하나의 전체 네트워크를 사용하여 이들의 예측 결합 효과를 근사한다. 일반적인 비선형 신경망에서 단일 네트워크의 출력이 모든 부분 네트워크 예측의 정확한 평균과 같다는 보장은 없다.

What each neuron computes

$$y_i^{(k)} = D\sigma\left(\sum_j w_{ji}^{(k)} y_j^{(k-1)} + b_i^{(k)}\right)$$

$$\mathrm{E}[y_i^{(k)}] = \alpha\sigma\left(\sum_j w_{ji}^{(k)} y_j^{(k-1)} + b_i^{(k)}\right)$$

- α : 뉴런이 살아남을 확률 (0\~1 사이의 고정값)
  $D \sim Bernoulli(\alpha)$: 확률 α로 1, 확률 $1-\alpha$로 0
  - D ⇒ (실제로 뉴런을 껐다킴)
- Training에서는 D를 곱해 뉴런을 무작위로 활성화/비활성화함.
  - → 뉴런 출력의 기댓값은 $E[y] = \alpha\sigma(z)$
    → **Inference에서는 Dropout을 끄고,**
    출력에 α를 곱해 기댓값을 사용함.
- ⇒ 여러 Sub-network의 Ensemble 효과를 근사하는 것.

$$W_{test} = \alpha W_{trained}$$